

# ARSITEKTUR & ALUR PENGIRIMAN EMAIL OTP VIA SMTP GMAIL

Implementasi Protokol RFC 5321 (Net::SMTP + STARTTLS), Manajemen Kredensial .env, Format Payload MIME HTML, dan Alur Otentikasi Terpadu DIJAK EXPRESS

Stack: Perl REST API (uWSGI / Mojolicious) • Enkripsi: TLS 1.3 / Port 587 • Google App Password 16-Digit • 2026

## RINGKASAN EKSEKUTIF LAYANAN EMAIL OTP

Layanan notifikasi email OTP pada sistem DIJAK EXPRESS menerapkan arsitektur terisolasi (Service Layer) berbasis pustaka Net::SMTP. Sistem ini mengamankan pengiriman kode 6-digit dengan enkripsi transport TLS (STARTTLS), otentikasi SASL dengan Google App Password, template HTML responsif bermerek (#FFD100), hashing SHA-256 pada database sebelum pengiriman, serta mode simulasi otomatis untuk lingkungan development offline.

## BAB 1: PENYIMPANAN DEPENDENSI & MANAJEMEN KREDENSIAL (STORAGE)

Berbeda dengan pemindai barcode yang berjalan di sisi peramban (client-side), layanan SMTP dieksekusi di lingkungan server backend (Perl runtime). Struktur dependensi terbagi menjadi modul pustaka sistem (CPAN) dan konfigurasi variabel lingkungan yang aman.

### 1.1. Modul Library Perl (CPAN)

Komunikasi protokol SMTP dikendalikan oleh modul CPAN standar industri yang memiliki kapabilitas enkripsi kriptografi tinggi:

| Nama Modul Perl | Tipe / Sumber    | Peran & Tanggung Jawab dalam Alur SMTP                                        |
|-----------------|------------------|-------------------------------------------------------------------------------|
| Net::SMTP       | Core / CPAN      | Klien protokol SMTP (RFC 5321), pembuka socket TCP, manajemen sesi            |
| IO::Socket::SSL | CPAN             | Penyedia socket terenkripsi TLS/SSL dan negosiasi jabat tangan sertifikat     |
| Authen::SASL    | CPAN             | Mekanisme otentikasi login PLAIN/LOGIN saat berinteraksi dengan server Google |
| Net::SSLeay     | CPAN (C-Binding) | Pustaka antarmuka level rendah ke engine OpenSSL sistem operasi               |

### 1.2. Penyimpanan Kredensial pada File .env

Kredensial email bersifat rahasia dan tidak boleh dicatat dalam repositori kode (hardcoded). Konfigurasi disimpan dalam file .env di root proyek:

| FILE KONFIGURASI LINGKUNGAN (.ENV)                                                                                                                                                                                                                                                                                    | ENV |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| <pre># .env - Konfigurasi SMTP Gmail SMTP_HOST=smtp.gmail.com SMTP_PORT=587 SMTP_USER=dijakexpress@gmail.com SMTP_PASSWORD=abcd efgh ijkl mnop      # 16-digit Google App Password (tanpa spasi) SMTP_SECURE=starttls                  # starttls (port 587) atau ssl (port 465) SMTP_FROM_NAME="DIJAK EXPRESS"</pre> |     |

### 1.3. Pembacaan Sentral pada backend/lib/Wahana/Config.pm

Modul Config.pm memuat variabel lingkungan dan menetapkan nilai default fallback yang aman jika konfigurasi belum disetel:

## CENTRAL CONFIGURATION MAPPING

PERL

```
# backend/lib/Wahana/Config.pm (Cuplikan Konfigurasi SMTP)
sub config {
    return {
        # ... konfigurasi database & JWT ...
        smtp_host      => $ENV{SMTP_HOST}      // 'smtp.gmail.com',
        smtp_port      => $ENV{SMTP_PORT}      // 587,
        smtp_user      => $ENV{SMTP_USER}      // '',
        smtp_pass      => $ENV{SMTP_PASSWORD}  // '',
        smtp_secure    => $ENV{SMTP_SECURE}    // 'starttls',
        smtp_from_name => $ENV{SMTP_FROM_NAME} // 'DIJAK EXPRESS',
    };
}
```

## BAB 2: MEKANISME PEMANGGILAN LIBRARY (INVOCATION & DESIGN)

Aplikasi memisahkan tugas pengiriman email ke dalam modul independen Wahana::Mail (Design Pattern: Service Helper). Pemisahan ini memastikan bahwa controller API hanya perlu memanggil satu fungsi sederhana tanpa perlu memahami detail protokol socket jaringan.

### 2.1. Pemanggilan oleh AuthController.pm

Saat pengguna meminta verifikasi login atau reset password, AuthController.pm membuat kode OTP acak, menyimpannya ke database, lalu mengeksekusi helper send\_otp\_email:

## PEMANGGILAN SERVICE DARI AUTH CONTROLLER

PERL

```
# backend/lib/Wahana/Controller/AuthController.pm
use Wahana::Mail qw(send_otp_email);

# 1. Generate OTP 6 digit dan hash SHA-256 untuk keamanan DB
my $otp_code = Wahana::Auth->generate_otp();
my $otp_hash = Wahana::Auth->hash_otp($otp_code);

# 2. Simpan hash ke database dengan masa berlaku 5 menit
$dbh->do(Wahana::Query->get('otp_insert'), undef, $user_id, $email, $otp_hash, $expires_at);

# 3. Panggil service pengiriman email via SMTP Gmail
my ($mail_ok, $mail_msg) = send_otp_email(
    to_email => $email,
    user_name => $user_row->{name},
    otp_code => $otp_code,
    context  => 'LOGIN',          # 'LOGIN' atau 'RESET_PASSWORD'
    app_name => 'DIJAK EXPRESS'
);

unless ($mail_ok) {
    return { success => \0, message => "Gagal mengirimkan OTP: $mail_msg" };
}
```

### 2.2. Validasi Format Email Masukan (Pre-Flight Sanitization)

Sebelum membuka koneksi socket ke server Google, pustaka memverifikasi integritas alamat email penerima menggunakan regex standar RFC 5322 untuk mencegah serangan header injection atau kegagalan transmisi:

## VALIDASI REGEX ALAMAT EMAIL

PERL

```
return (0, "Alamat email penerima tidak valid.")
    unless defined $to_email && $to_email =~ /^[\^s@]+@[\^s@]+\.[\^s@]+$/;
```

## BAB 3: PROTOKOL NEGOSIASI ENKRIPSI & JABAT TANGAN JARINGAN (HANDSHAKE)

Server SMTP Google (smtp.gmail.com) menuntut standar keamanan modern. Koneksi tidak diizinkan mengirimkan pesan plaintext tanpa melalui enkripsi kriptografi Transport Layer Security (TLS).

### 3.1. Perbandingan Port 587 (STARTTLS) vs Port 465 (Direct SSL)

| Parameter        | Port 587 (STARTTLS - Direkomendasikan)                          | Port 465 (Direct SSL / SMTPS)                                   |
|------------------|-----------------------------------------------------------------|-----------------------------------------------------------------|
| Inisiasi Soket   | Koneksi dibuka dalam TCP biasa, lalu di-upgrade via STARTTLS    | Koneksi langsung terenkripsi SSL/TLS sejak byte pertama dibuka  |
| Kompatibilitas   | Standar resmi IETF untuk submission surat elektronik (RFC 6409) | Protokol legacy yang didukung Google Cloud / VPS tertentu       |
| Konfigurasi .env | SMTP_PORT=587, SMTP_SECURE=starttls                             | SMTP_PORT=465, SMTP_SECURE=ssl                                  |
| Kelebihan        | Lebih fleksibel menembus firewall jaringan ISP kantor/gudang    | Sedikit lebih cepat pada handshake awal jika SSL didukung penuh |

### 3.2. Urutan Perintah Jaringan (Low-Level SMTP Dialog)

- 1. **TCP Connect:** Klien membuka soket TCP ke smtp.gmail.com:587. Server membalas: 220 smtp.gmail.com ESMTP.
- 2. **EHLO Negosiasi:** Klien mengirim EHLO localhost. Google membalas dengan kapabilitas server (250-STARTTLS, 250-AUTH).
- 3. **Upgrade STARTTLS:** Klien mengirim perintah STARTTLS. Google membalas 220 2.0.0 Ready to start TLS.
- 4. **TLS Handshake:** Klien dan Google menegosiasikan enkripsi TLS 1.3 melalui pustaka IO::Socket::SSL.
- 5. **SASL Authentication:** Klien mengirim kredensial terenkripsi base64 (\$smtp->auth). Google membalas 235 2.7.0 Accepted.

IMPLEMENTASI KONEKSI, STARTTLS & AUTENTIKASIPERL

```
# backend/lib/Wahana/Mail.pm (Cuplikan Jabat Tangan Koneksi)
my %smtp_opts = ( Port => $port, Timeout => 20, Debug => 0 );

if ($is_direct_ssl) {
    $smtp_opts{SSL} = 1;
    $smtp_opts{SSL_verify_mode} = 0;
}

my $smtp = Net::SMTP->new($host, %smtp_opts);
die "Gagal terhubung ke $host:$port ($!)" unless $smtp;

# Upgrade ke TLS jika menggunakan port 587
if (!$is_direct_ssl && ($port == 587 || $sec_mode eq 'starttls')) {
    if ($smtp->can('starttls')) {
        $smtp->starttls(SSL_verify_mode => 0) or die "STARTTLS gagal diinisialisasi";
    }
}

# Otentikasi SASL dengan Google App Password
my $auth_ok = eval { $smtp->auth($user, $pass) };
die "Autentikasi gagal: $_" unless $auth_ok;
```

## BAB 4: TRANSMISI ENVELOPE & FORMAT PAYLOAD MIME HTML

Setelah otentikasi berhasil, pesan dikirimkan menggunakan protokol surat elektronik berbasis amplop (envelope) dan struktur konten MIME (Multipurpose Internet Mail Extensions) bertipe text/html.

## 4.1. Pemisahan Envelope dan Konten Pesan

Pustaka Net::SMTP mengeksekusi urutan pengalamatan sebelum memasukkan data visual email:

- **Amplop Pengirim (MAIL FROM):** \$smtp->mail(\$user) mengabarkan server alamat akun asal.
- **Amplop Penerima (RCPT TO):** \$smtp->to(\$to\_email) menentukan tujuan akhir email.
- **Header Data (DATA):** Menyusun header From, To, Subject, MIME-Version, dan Content-Type.

## 4.2. Struktur Template HTML Bermerek DIJAK EXPRESS

Email dibuat menggunakan tata letak tabel HTML yang kompatibel dengan seluruh aplikasi email (Gmail Mobile, Apple Mail, Outlook, dan Webmail) tanpa terdistorsi:

### TRANSMISI PAYLOAD DATA & TEMPLATE HTML RESPONSIVE

[PERL](#)

```
$smtp->data();
$smtp->datasend("From: $from_name <$user>\n");
$smtp->datasend("To: $to_email\n");
$smtp->datasend("Subject: $subject\n");
$smtp->datasend("MIME-Version: 1.0\n");
$smtp->datasend("Content-Type: text/html; charset=UTF-8\n\n");

# Body HTML Responsif
$smtp->datasend(<<"END_HTML");
<!DOCTYPE html>
<html>
<body style="margin:0; padding:0; background-color:#f4f6f9; font-family:Arial,sans-serif;">
  <table width="100%" cellpadding="0" cellspacing="0" style="padding:30px 10px;">
    <tr><td align="center">
      <table width="100%" style="max-width:580px; background:#ffffff; border-radius:12px; overflow:hidden;">
        <!-- Header Kuning Khas DIJAK EXPRESS -->
        <tr><td style="background-color:#FFD100; padding:24px 32px;">
          <div style="font-size:22px; font-weight:bold; color:#111111;">DIJAK EXPRESS</div>
          <div style="font-size:12px; color:#333333;">Sistem Informasi Logistik</div>
        </td></tr>
        <!-- Kontainer Kode OTP 6-Digit -->
        <tr><td style="padding:32px;">
          <h2 style="color:#111111;">$title</h2>
          <p style="color:#333333;">$explanation</p>
          <div style="background:#f8fafc; border:2px dashed #FFD100; border-radius:8px; padding:18px; text-align:center;">
            <span style="font-family:monospace; font-size:36px; font-weight:bold; letter-spacing:8px; color:#0b2341;">$otp_code
          </span>
          </div>
          <p style="font-size:12px; color:#dc2626; margin-top:20px;">
            Kode berlaku selama 5 menit. Jangan berikan kode ini kepada siapa pun!
          </p>
        </td></tr>
      </table>
    </td></tr>
  </table>
</body>
</html>
END_HTML
);

$smtp->dataend(); # Menutup transmisi pesan dengan tanda titik (CRLF . CRLF)
$smtp->quit();    # Menutup sesi koneksi soket dengan server Google
```

## BAB 5: SIKLUS HIDUP VERIFIKASI OTP END-TO-END (LIFECYCLE)

Integrasi SMTP Gmail merupakan bagian dari arsitektur Otentikasi Dua Faktor (2FA) dan Pemulihan Akun yang menghubungkan antarmuka pengguna Vue 3 dengan logika bisnis backend Perl.

### 5.1. Alur Lengkap Verifikasi Login Petugas

- **Tahap 1 (Validasi Kredensial):** Petugas memasukkan email dan kata sandi pada LoginPage.vue. Backend memverifikasi hash bcrypt.
- **Tahap 2 (Generasi & Enkripsi DB):** Backend menghasilkan 6 angka acak (misal: 849201). Angka ini di-hash dengan SHA-256 lalu disimpan ke tabel user\_otps bersama masa kedaluwarsa 5 menit.
- **Tahap 3 (Dispatching SMTP):** Angka mentah 849201 dikirimkan ke kotak masuk Gmail petugas melalui Wahana::Mail. Plaintext OTP TIDAK disimpan di database demi mematuhi asas zero-knowledge.
- **Tahap 4 (Pre-Auth Challenge):** Backend mengembalikan token sementara (preauth\_token) dan flag require\_otp: true ke frontend.
- **Tahap 5 (Verifikasi Masukan):** Petugas memasukkan 6 angka dari email. Backend meng-hash masukan petugas lalu mencocokkannya dengan hash di tabel user\_otps.
- **Tahap 6 (Penerbitan JWT):** Jika cocok dan belum kedaluwarsa, OTP ditandai terpakai (is\_used = 1) dan sistem menerbitkan JWT Access Token resmi.

### 5.2. Perlindungan Keamanan & Pencegahan Abuse (Security Hardening)

#### MEKANISME PENGAMANAN ANTI-BRUTE FORCE & COOLDOWN RESEND

1. Masa Berlaku Ketat (5 Menit): Record pada tabel user\_otps memiliki kolom expires\_at. Permintaan setelah 5 menit otomatis ditolak.
2. Cooldown Resend (60 Detik): Tombol "Kirim Ulang OTP" di frontend dikunci timer 60 detik untuk mencegah flooding server SMTP.
3. Invalidation OTP Lama: Saat pengiriman ulang diminta, seluruh kode OTP aktif sebelumnya untuk email tersebut langsung dibatalkan (is\_used = 1).
4. Penyimpanan Terenkripsi: Database hanya menyimpan hash SHA-256 OTP sehingga jika terjadi data leak, kode OTP tidak dapat dibaca.

## BAB 6: TROUBLESHOOTING, MODE SIMULASI & PANDUAN SETUP GOOGLE

Untuk menjaga stabilitas pengembangan serta mencegah terhentinya sistem di lingkungan tanpa koneksi internet, sistem dirancang dengan fail-safe otomatis.

### 6.1. Fail-Safe: Mode Simulasi Otomatis (Simulation Mode)

Jika variabel SMTP\_USER atau SMTP\_PASSWORD pada file .env belum diisi atau bernilai default, sistem tidak akan mengalami error/crash, melainkan beralih ke Mode Simulasi:

#### AUTOMATIC SIMULATION FALLBACK

PERL

```
# backend/lib/Wahana/Mail.pm (Cuplikan Mode Simulasi)
if (!$user || !$pass || $pass eq 'ganti-dengan-app-password') {
    warn "[MAIL SIMULATION] Kredensial SMTP belum disetel di .env ($user). " .
        "Simulasi pengiriman OTP ke $to_email BERHASIL.\n";
    return (1, "Kode OTP berhasil dikirim (mode simulasi SMTP).");
}
```

Mode ini memungkinkan pengujian alur fungsional aplikasi (automated test / dev offline) tetap berjalan lancar karena kode OTP tetap tercatat di database dan dapat dibaca melalui log console server.

6.2. Panduan Pembuatan Google App Password (16-Digit)

- **Langkah 1:** Buka akun Google yang akan dijadikan pengirim notifikasi: <https://myaccount.google.com/>
- **Langkah 2:** Masuk ke menu Keamanan (Security) dan pastikan Verifikasi 2 Langkah (2-Step Verification) sudah AKTIF.
- **Langkah 3:** Ketik "Sandi Aplikasi" atau "App Passwords" pada kolom pencarian pengaturan akun Google.
- **Langkah 4:** Buat sandi baru dengan nama aplikasi "DIJAK Express Backend".
- **Langkah 5:** Google akan menampilkan 16 karakter unik (contoh: abcd efgh ijkl mnop). Salin kode tersebut ke file .env tanpa spasi.

6.3. Pemetaan Kode Masalah & Solusi Lapangan (Error Matrix)

| Pesan Error Log                                  | Akar Masalah Teknis                                              | Langkah Solusi                                                                          |
|--------------------------------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Connection timed out / Connection refused        | Port 587 atau 465 diblokir firewall hosting / ISP lokal          | Buka port outbound 587 pada firewall server atau ubah port ke 465 dengan mode SSL       |
| 535 5.7.8 BadCredentials / Username not accepted | Menggunakan password login biasa atau salah ketik App Password   | Wajib gunakan 16-digit Google App Password, pastikan akun Google telah mengaktifkan 2FA |
| STARTTLS failed / Cannot upgrade socket          | Modul IO::Socket::SSL atau Net::SSLeay belum terpasang di OS     | Jalankan cpanm IO::Socket::SSL atau install package libio-socket-ssl-perl via apt       |
| 550 5.1.1 Recipient Rejected / User unknown      | Alamat email tujuan salah atau domain email tidak menerima surat | Periksa format email penerima pada database atau pastikan email penerima aktif          |

DIJAK EXPRESS — STANDARISASI SISTEM EMAIL NOTIFIKASI

Dokumen ini menjadi standar resmi implementasi layanan pengiriman email OTP, verifikasi akun, dan pemulihan kata sandi pada ekosistem aplikasi DIJAK Express.